Practice · ++, -- and +=

Each question lists its options; the right one is marked **Right answer**. *Feedback* is what students see after they answer. In a typed question the blank is in square brackets, with the accepted answers separated by `|`.

(mc) L1 · What i++ means

What does `i++;` do?

- It prints `i`.
- It adds `i` to itself.
- **Right answer:** It adds 1 to `i`. It is the short form of `i = i + 1;`.
- It checks whether `i` is positive.

Feedback: `i++;` means "make `i` one bigger". `i--;` means "make `i` one smaller". On a line of their own, `i++;` and `++i;` do exactly the same thing (class 5, slide 71).

(mc) L1 · What += means

Which line does the same as `x += 3;`?

- `x = 3;`
- **Right answer:** `x = x + 3;`
- `x + 3;`
- `x == x + 3;`

Feedback: `x += 3;` takes the value in the box `x`, adds 3, and puts the result back into `x`. The same idea works for the others: `x -= 2;` is `x = x - 2;`, `x *= 4;` is `x = x * 4;`, `x /= 5;` is `x = x / 5;`, `x %= 2;` is `x = x % 2;` (class 5, slide 84).

(mc) L1 · The best use of +=

```
int sum = 0;
for (int i = 1; i <= 4; i++) {
    sum += i;
}
printf("%d", sum);
```

What does `sum += i;` do on each repeat?

- It sets `sum` to `i`.
- **Right answer:** It adds the current `i` to the total kept in `sum`.
- It adds 1 to `sum`.
- It adds `sum` to `i`.

Feedback: `sum` keeps a running total: 0, then 1, 3, 6, 10. Each repeat adds the current `i`. The program prints `10`. Adding into a total is the most common use of `+=`.

(mc) L2 · Use, then add

```
int x = 5;
printf("%d ", x++);
printf("%d", x);
```

What is printed?

- `5 5`
- **Right answer:** `5 6`
- `6 6`
- `6 7`

Feedback: `x++` inside a bigger line means: use the **current** value first, then add 1. So the first `printf` shows 5, and after it `x` is 6. The second `printf` shows 6. This is Q2 on class 6, slide 92.

(mc) L2 · Add, then use

```
int a = 5;
int b = ++a;
printf("a=%d b=%d", a, b);
```

What is printed?

- `a=5 b=5`
- `a=6 b=5`
- **Right answer:** `a=6 b=6`
- `a=5 b=6`

Feedback: `++a` means: add 1 first, then use the new value. So `a` becomes 6 and `b` gets 6. With `int b = a++;` you would get `a=6 b=5`, because `b` gets the old value (class 5, slides 77 and 78). To stay safe, write `i++;` on its own line and do not mix it into bigger lines.

(mc) L2 · A chain of updates

```
int x = 10;
x += 3;
x -= 2;
x *= 4;
x /= 5;
printf("%d", x);
```

What is printed?

- `44`
- `9`
- **Right answer:** `8`
- `8.8`

Feedback: Go line by line: 10 + 3 is 13, 13 - 2 is 11, 11 * 4 is 44, and 44 / 5 is 8, because `x` is an `int` and integer division drops the decimal part. One more line, `x %= 2;`, would make it 0.

(fib) L3 · Type the short form

Write the short form of `count = count + 1;` with `++`. Type it exactly, with the semicolon and no spaces.

[count++;|++count;]

Feedback: `count++;` adds 1 to `count`. You will write this line in almost every loop.

(fib) L3 · Type the operator

Complete the line so it takes 30 away from `balance`. Type only the two-character operator.

```
int balance = 100;
balance [-=] 30;
printf("%d", balance);
```

Feedback: `balance -= 30;` is the short form of `balance = balance - 30;`. The program prints `70`.

(fib) L3 · What is in the box

What does this print? Type only the number.

```
int t = 17;
t %= 5;
printf("%d", t);
```

It prints [2].

Feedback: `t %= 5;` is `t = t % 5;`. The remainder of 17 divided by 5 is 2, because 3 * 5 is 15 and 2 is left over.